Serverless Image Compression

using AWS Lambda, Amazon S3 and Python Pillow

Project Documentation

Item	Value
Author	Vanticheruku Mohith Reddy
GitHub	github.com/mohithreddy894-sys
Lambda function	image-compresser
Input / Output buckets	input-0-m6 / output-0-m6
Technologies	AWS Lambda, Amazon S3, IAM, CloudWatch, Python, Boto3, Pillow

1. Project Overview

This project automatically compresses images using AWS Lambda, Amazon S3, Python and the Pillow library. An image is uploaded to the input S3 bucket. The Lambda function retrieves the image, compresses it with Pillow using JPEG quality 50, and stores the compressed image in a separate output S3 bucket.

The project uses a serverless architecture, so there is no need to manage EC2 servers.

Item	Value
Input bucket	input-0-m6
Output bucket	output-0-m6
Lambda function	image-compresser
Runtime	Python
Compression	JPEG, quality 50, optimize=True
Output naming	compressed-<original-name>.jpeg

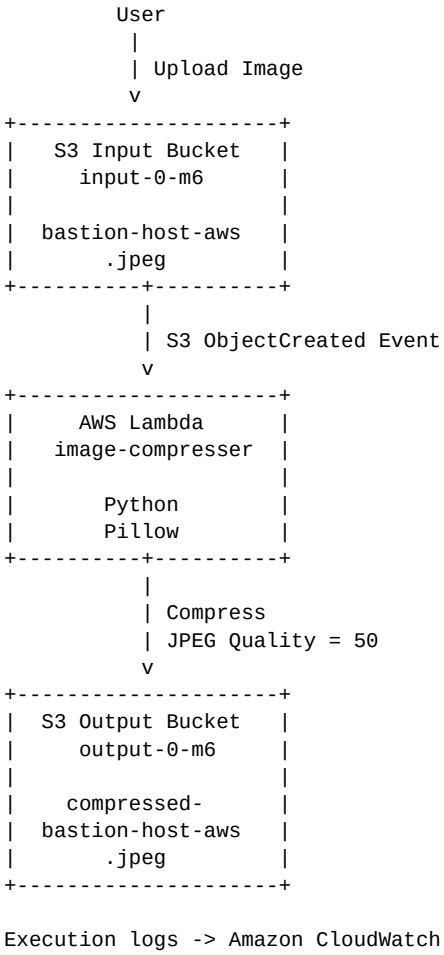
2. Objectives

- Automatically process images uploaded to Amazon S3.
- Reduce image file size using compression.
- Convert images into JPEG format.
- Store the compressed image separately from the original.
- Use AWS Lambda for serverless image processing.
- Implement proper IAM permissions for secure S3 access.

3. AWS Services and Tools Used

Service / Tool	Purpose
Amazon S3	Stores original and compressed images
AWS Lambda	Performs the image compression
AWS IAM	Gives Lambda permission to access S3
Amazon CloudWatch Logs	Monitors Lambda execution
Python	Programming language
Boto3	AWS SDK for Python (S3 access)
Pillow	Image processing / compression library

4. Architecture



5. Project Workflow

Step 1: Upload Image

The original image is uploaded to the input bucket **input-0-m6**, for example **bastion-host-aws.jpeg**.

Step 2: Lambda Receives the Event

When the Lambda function is connected to an S3 **ObjectCreated** trigger, S3 sends information about the uploaded object to Lambda. The event contains:

```
Bucket = input-0-m6
Key    = bastion-host-aws.jpeg
```

Step 3: Lambda Downloads the Image

Python's Boto3 library retrieves the image:

```
response = s3.get_object(
    Bucket=bucket,
    Key=key
)
```

Step 4: Pillow Opens the Image

The downloaded image is loaded into memory:

```
image = Image.open(io.BytesIO(image_content))
```

Step 5: Convert to RGB

JPEG does not support some image modes such as RGBA, so the code converts the image when needed:

```
if image.mode != "RGB":
    image = image.convert("RGB")
```

Step 6: Compress the Image

The image is saved with JPEG quality 50:

```
image.save(
    buffer,
    format="JPEG",
    quality=50,
    optimize=True
)
```

Step 7: Upload the Compressed Image

The compressed image is uploaded to **output-0-m6** with the file name **compressed-bastion-host-aws.jpeg**.

6. Lambda Source Code

```
import boto3
from PIL import Image
import io
import urllib.parse

s3 = boto3.client("s3")

OUTPUT_BUCKET = "output-0-m6"


def lambda_handler(event, context):

    bucket = event["Records"][0]["s3"]["bucket"]["name"]

    key = urllib.parse.unquote_plus(
        event["Records"][0]["s3"]["object"]["key"]
    )

    print("Input bucket:", bucket)
    print("Input key:", key)

    response = s3.get_object(
        Bucket=bucket,
        Key=key
    )

    image_content = response["Body"].read()

    image = Image.open(io.BytesIO(image_content))

    print("Original format:", image.format)
    print("Original size:", image.size)
    print("Original mode:", image.mode)

    if image.mode != "RGB":
        image = image.convert("RGB")

    buffer = io.BytesIO()

    image.save(
        buffer,
        format="JPEG",
        quality=50,
```

```
        optimize=True
    )

    buffer.seek(0)

    filename = key.split("/")[-1]

    filename_without_extension = filename.rsplit(".", 1)[0]

    output_key = (
        "compressed-"
        + filename_without_extension
        + ".jpeg"
    )

    print("Output bucket:", OUTPUT_BUCKET)
    print("Output key:", output_key)

    s3.put_object(
        Bucket=OUTPUT_BUCKET,
        Key=output_key,
        Body=buffer.getvalue(),
        ContentType="image/jpeg"
    )

    print("Image compression completed successfully.")

    return {
        "statusCode": 200,
        "message": "Image compressed successfully",
        "input_bucket": bucket,
        "input_key": key,
        "output_bucket": OUTPUT_BUCKET,
        "output_key": output_key
    }
```

7. IAM Configuration

The Lambda execution role (**image-compresser-role-m3kpga5s**) needs permission to read objects from the input bucket, list the input bucket, and write objects to the output bucket. It also needs the standard CloudWatch Logs permissions (for example the AWSLambdaBasicExecutionRole managed policy) so that logs are delivered.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadInputBucket",
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket"
      ],
      "Resource": "arn:aws:s3:::input-0-m6"
    },
    {
      "Sid": "ReadInputObjects",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::input-0-m6/*"
    },
    {
      "Sid": "WriteOutputObjects",
      "Effect": "Allow",
      "Action": [
        "s3:PutObject"
```

```
      ],
      "Resource": "arn:aws:s3:::output-0-m6/*"
    }
  ]
}
```

Permission Flow

Permission	Resource	Purpose
s3:ListBucket	input-0-m6	List the input bucket
s3:GetObject	input-0-m6/*	Download the original image
s3:PutObject	output-0-m6/*	Upload the compressed image

8. Pillow Lambda Layer

The AWS Lambda Python runtime does not automatically contain Pillow. Therefore, a Lambda Layer containing Pillow was attached to the Lambda function. The function imports it using:

```
from PIL import Image
```

Without the Pillow layer, the Lambda would typically fail with:

```
ModuleNotFoundError: No module named 'PIL'
```

The layer's Python version and CPU architecture (x86_64 or arm64) must match the Lambda function's configuration.

9. Lambda Test Event

For the test image **bastion-host-aws.jpeg**, the test event is:

```
{
  "Records": [
    {
      "s3": {
        "bucket": {
          "name": "input-0-m6"
        },
        "object": {
          "key": "bastion-host-aws.jpeg"
        }
      }
    }
  ]
}
```

10. Test Result

The successful Lambda execution produced the following log output:

```
Input bucket:    input-0-m6
Input key:       bastion-host-aws.jpeg
Original format: JPEG
Original size:   (1536, 1024)
Original mode:   RGB
Output bucket:   output-0-m6
Output key:      compressed-bastion-host-aws.jpeg
Image compression completed successfully.
```

Lambda response:

```
{
  "statusCode": 200,
  "message": "Image compressed successfully",
  "input_bucket": "input-0-m6",
  "input_key": "bastion-host-aws.jpeg",
  "output_bucket": "output-0-m6",
  "output_key": "compressed-bastion-host-aws.jpeg"
}
```

11. Error Handling During Development

Error 1: AccessDenied

Initially, Lambda did not have permission to access the input S3 bucket.

```
not authorized to perform: s3:ListBucket
```

Solution: added **s3:ListBucket** and **s3:GetObject** to the Lambda execution role.

Error 2: NoSuchKey

The test event contained an incorrect object key (**test.jpeg**) while the actual S3 object was **bastion-host-aws.jpeg**.

Solution: used the exact S3 object key in the test event.

```
"key": "bastion-host-aws.jpeg"
```

Error 3: AllAccessDisabled

The Lambda was initially configured with the wrong output bucket (**output-bucket-m4**) while the actual bucket was **output-0-m6**.

Solution: changed the bucket name in the code and updated the IAM policy accordingly.

```
OUTPUT_BUCKET = "output-0-m6"
```

12. Final Project Structure

```
AWS Image Compression Project
|
+-- Amazon S3
|   |
|   +-- input-0-m6
|   |   +-- bastion-host-aws.jpeg
|   |
|   +-- output-0-m6
|       +-- compressed-bastion-host-aws.jpeg
|
+-- AWS Lambda
|   +-- image-compresser
|       +-- lambda_function.py
|
+-- Lambda Layer
|   +-- Pillow
|
+-- IAM
|   +-- image-compresser-role-m3kpga5s
|
+-- CloudWatch
    +-- Lambda execution logs
```

13. Key Concepts Demonstrated

- Serverless, event-driven architecture
- S3 object storage and S3 event notifications (ObjectCreated)
- IAM roles and least-privilege policies (s3:GetObject, s3:PutObject, s3:ListBucket)
- Python with Boto3 for AWS automation
- Image processing and JPEG compression with Pillow
- Lambda Layers for third-party dependencies
- Monitoring and debugging with CloudWatch Logs

14. Future Improvements

- Process every record in the event instead of only the first one.
- Add try/except error handling and an alert (SNS or dead-letter queue) on failure.
- Make the compression quality and output bucket configurable through environment variables.
- Support additional formats (PNG, WebP) and resizing / thumbnails.
- Define the whole stack as code with AWS SAM or Terraform.
- Add S3 lifecycle rules to expire old images and control storage cost.

15. Resume Summary

Serverless Image Compression | AWS Lambda, S3, IAM, Python, Pillow

- Built an event-driven pipeline where images uploaded to S3 automatically trigger a Lambda function that compresses them to JPEG (quality 50) and stores them in a separate output bucket.
- Configured a least-privilege IAM role (s3:GetObject, s3:ListBucket, s3:PutObject) and attached a Pillow Lambda layer to package the image-processing dependency.
- Debugged AccessDenied, NoSuchKey and misconfigured-bucket errors using CloudWatch Logs and IAM policy analysis.

16. Interview Talking Points

- **Why serverless?** No servers to patch or scale, and you pay only when images are processed.
- **Why two buckets?** Writing output back to the input bucket would re-trigger the Lambda and could cause an infinite loop.
- **Why a Lambda layer?** Pillow is not in the default runtime, and a layer keeps the deployment package small and reusable.
- **Why convert to RGB?** JPEG has no alpha channel, so modes like RGBA or P must be converted before saving.
- **Why unquote_plus on the key?** S3 event keys are URL-encoded, so spaces arrive as + and special characters as %XX.
- **Why s3:ListBucket?** Without it, a missing object returns AccessDenied instead of NoSuchKey, which makes debugging harder.